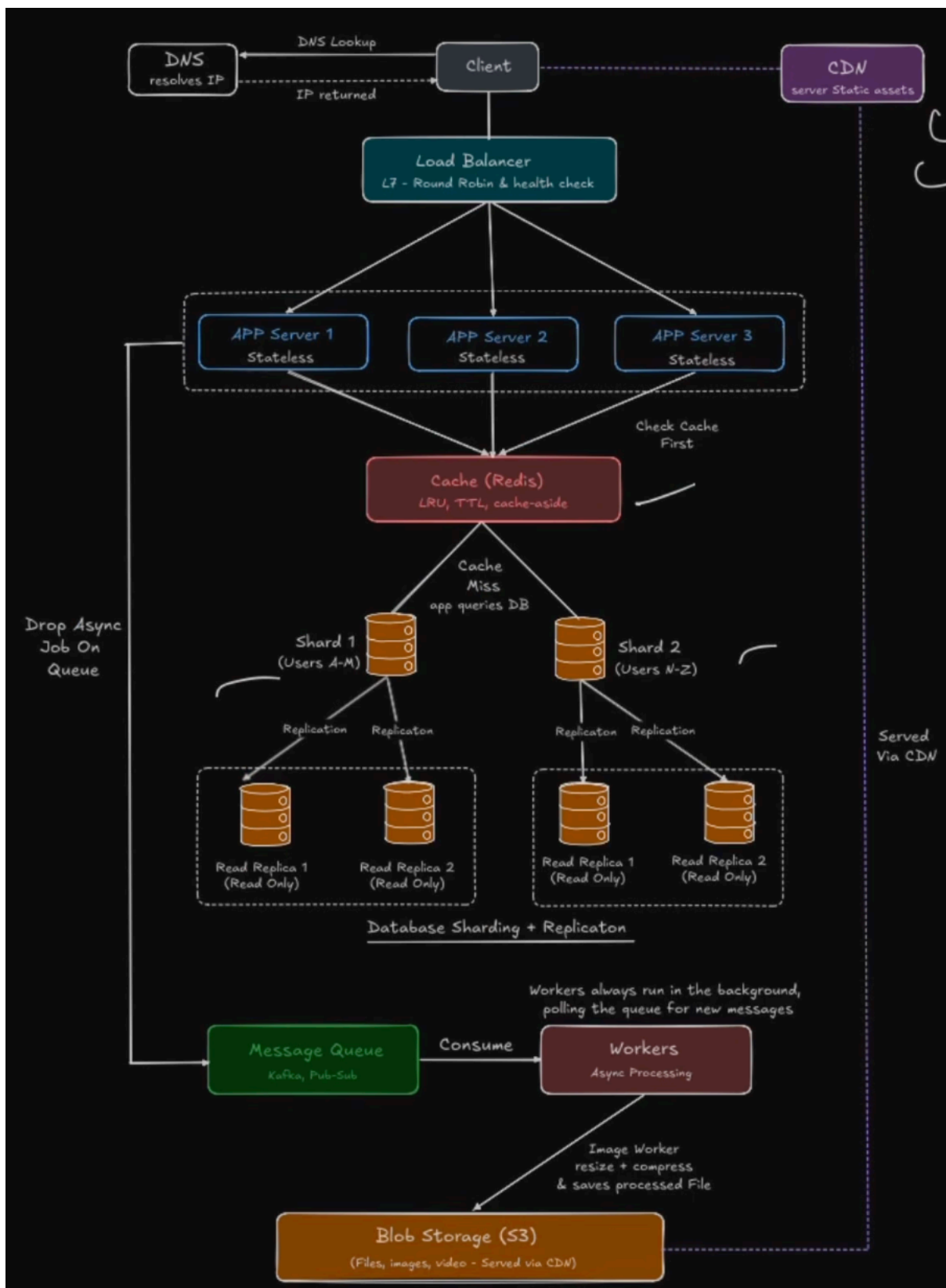


System Design - Terminologies

Roadmap:

1. *Foundation* - what, computation, components, functional and non functional aspects
2. *Communication* - DNS, API's, http, RESTful suite
3. *Data Layer* - Sql, no Sql, Cache
4. *Scaling and distribution* - CAP, load balancer, replication, partitioning
5. *Reliability and Operations* - Message queues, fault tolerance, Monitoring
6. *Case Studies* - in different notes.



Analogy to Cashier-Customer:

How can we improve this ?

- Improve skills of cashier.
- Suppose cashier has 3 tasks (money, receipt, checkout) - we must delegate these tasks to different people.
- Increase number of cashiers and their delegations as well - but in this database must be central
- balancing load on all the cash counters - equally spread in ideal case.

Mapping to system design:

- Improve code quality - TC and SC, maintainability, extensibility - LLD and DSA.
- Vertical Scaling - improving the single server that we have.
- Horizontal Scaling - adding more servers.
- Load balancers - delegating the tasks to each server properly.

Types of Architecture:

1. Monolith - one app deployed as one unit, fast in process calls, easier debugging, difficult to scale.
2. Distributed / micro services - independently deployed services, more caching, clear team ownership.

Synchronous and Asynchronous calls:

if we wait for some service to complete and respond back - Synchronous call.

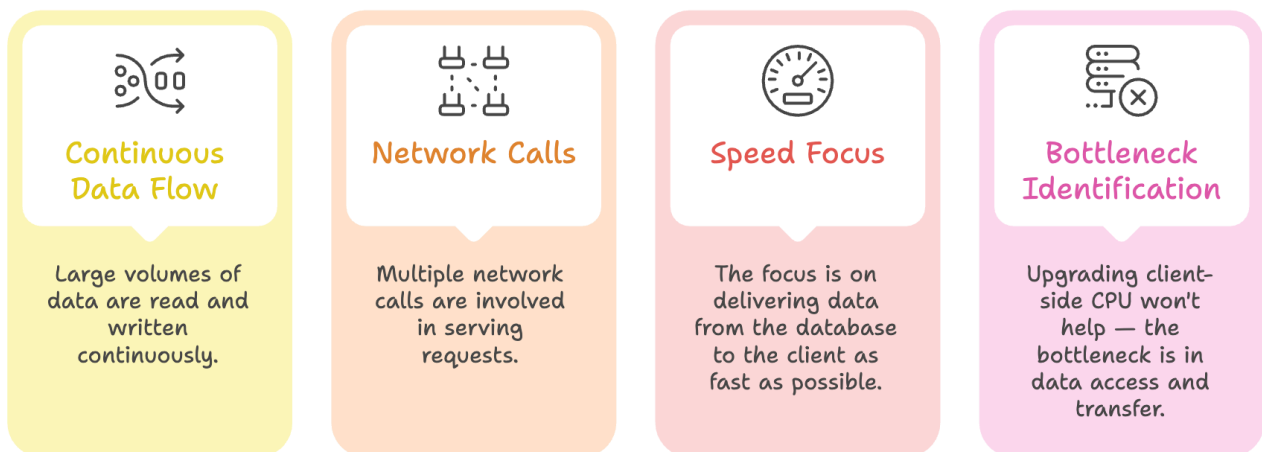
if we do not wait for some service to complete and respond back - Asynchronous call.

Message queues are used for handling asynchronous tasks.

Types of Applications: (Both have latency issue - what we have to fix)

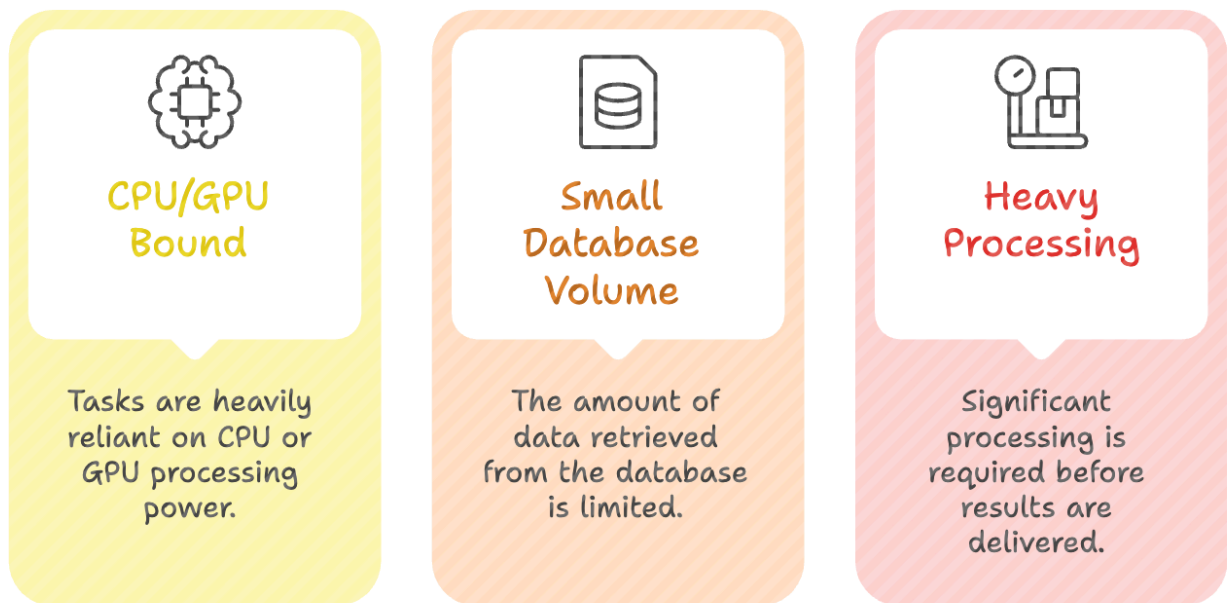
1. Data Intensive Apps - less maths, more fetch calls from databases - optimisations in Disk I/O, Network calls, Memory.
2. Compute Intensive Apps - more maths, less data from databases - GPU bounds - optimisations in GPU/CPU.

Data-Intensive Application Characteristics



Solved by - caching, Sharding, replication, consistency, DB optimisations.

Compute-Intensive Application Characteristics



eg - image processing, ML Model training, Cryptography, video rendering etc.
Solved by - Parallelism, GPU acceleration, algorithm optimisation

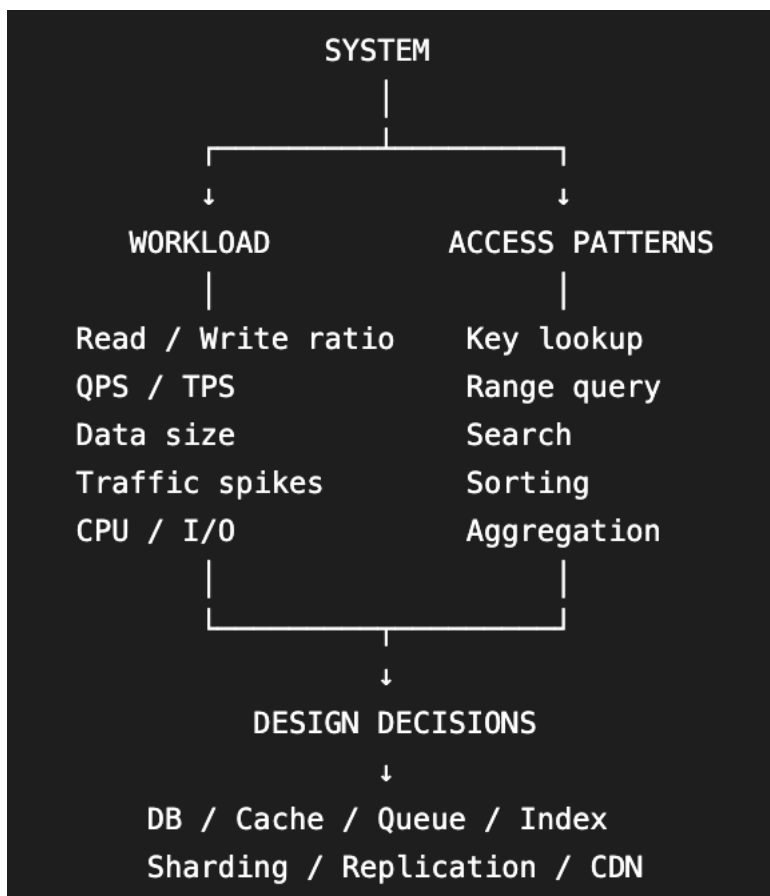
Types of Requirements:

1. Functional - user functions, features of app.
2. Non-Functional - uptime defined in SLA/SLO, scalability, fault tolerance, estimations of numbers, auto scaling, security, latency, authorisation, maintainability, reliability, observability.

Workloads and Access Patterns:

Workload - what type of tasks is system doing for the most time.

Access Patterns - how is the data being fetched.



Workload	Meaning	Example
Read-heavy	Much more reading than writing	Instagram feed, news website
Write-heavy	Much more writing than reading	Logging system, analytics ingestion
Read-write balanced	Reads and writes are comparable	E-commerce
Compute-heavy	Requires significant CPU/GPU processing	ML inference, video encoding
I/O-heavy	Lots of disk/network operations	File storage
Memory-heavy	Large amount of frequently accessed data	Caching, recommendation systems

How do you know that your design is good enough?

1. Have you divided system into components?
2. Every component has clear set of responsibilities?
3. Figure out details like database, caching, fault tolerance, async/sync communication.
4. Is each component in isolation scalable, fault tolerant and reliable? No SINGLE POINT OF FAILURE (SPOF).

Approach Question:

Clarify Requirements - who is using system, core use case, exclusions, available service, durable service.

Estimate Scale - average and peak QPS, read/write scale, latency sensitivity operations, geography, constraints.

Sketch APIs - data flow, authorisations, pagination, filters.

Design Simple System - entities and access patterns.

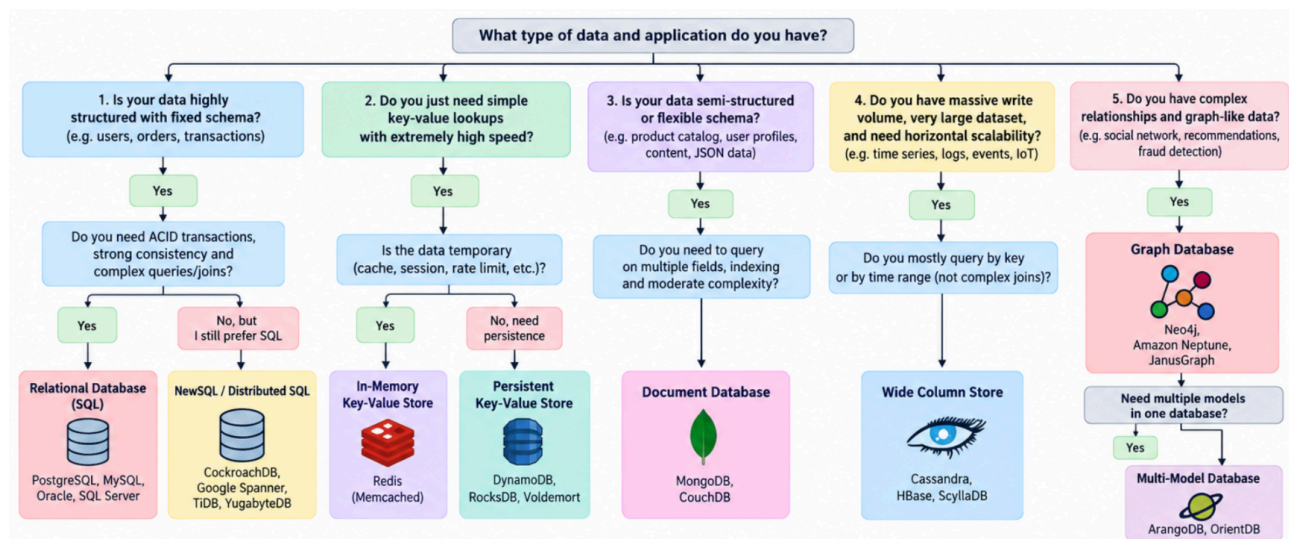
Find Bottlenecks - find bottlenecks like DB - scale DB then.

Add Justified mechanisms -

Check Tradeoffs -

1. define actors, core use cases, exclusions
2. clarify the quality requirements and the constraints
3. identify core entities and access patterns
4. design high levels API and very imp data flows
5. start with very simple high level HLD
6. estimate traffic, stroage, bandwidth, concurrency -> help you to make a decision
7. find the bottlenecks and try to implement the solution from it and also discuss the trade off
8. calculate avg qps, peak qps, read vs write ratio, stroage growth - if applicable to that question
9. bandwidth, data transfer, etc

Which database to use?



Domain Name System:

We cannot store all the websites domain names map to IP addresses in our systems or browsers as the data is too large so we have DNS for resolving this issue.

Issues:

1. Storage
2. Modification of IP address maintenance.

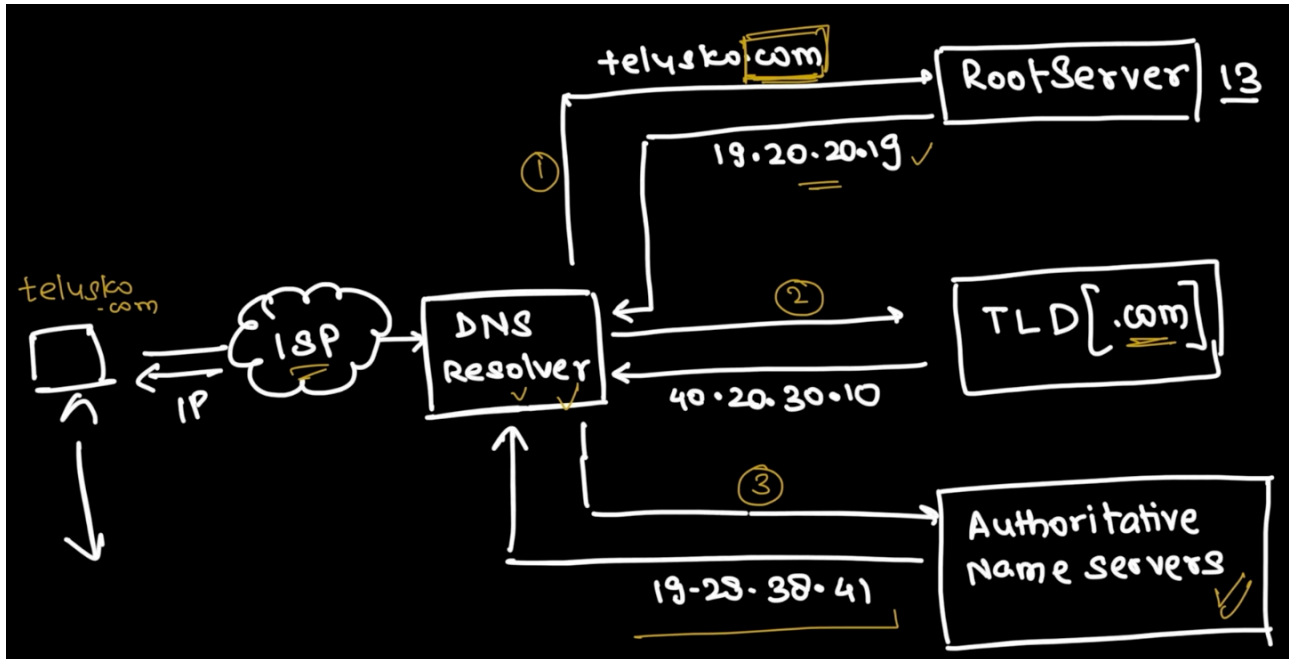
DNS Resolution Process



Steps are as follows:

1. Your machine connects to the internet through an Internet Service Provider (ISP), which has its own DNS resolver.
2. There are 13 root servers worldwide, named a.root-server.net through m.root-server.net

3. The root server does not know the final IP address - it directs the resolver to the appropriate TLD server.
4. TLD are top level domains example - .com, .net, .in, .co.in etc.
5. The TLD server responds with the IP address of the Authoritative Name Server that manages the specific domain.
6. ANS holds the actual DNS records for the domain.
7. Returns the final IP address of the requested domain to the DNS Resolver.
8. The DNS Resolver returns the IP address to the browser.



DNS Resolution Process



User Input

The user types the website address into their browser (telusko.com)

The user's ISP's DNS resolver receives the request

DNS Resolver



Root Server

The DNS resolver queries the root server for the .com TLD server's IP address (a-m.root-server.net)

The root server returns the IP address of the .com TLD server

TLD Server



Authoritative Name Server

The TLD server returns the IP address of the authoritative name server (telusko.com)

The browser connects to the IP address of telusko.com, and the webpage loads

Browser Connection



Other terms related to network that are essential for system design are:

1. IP Address and Ports
2. TCP/UDP
3. HTTP/HTTPS and TLS
4. Reverse Proxy - receives request and routes to backed server.

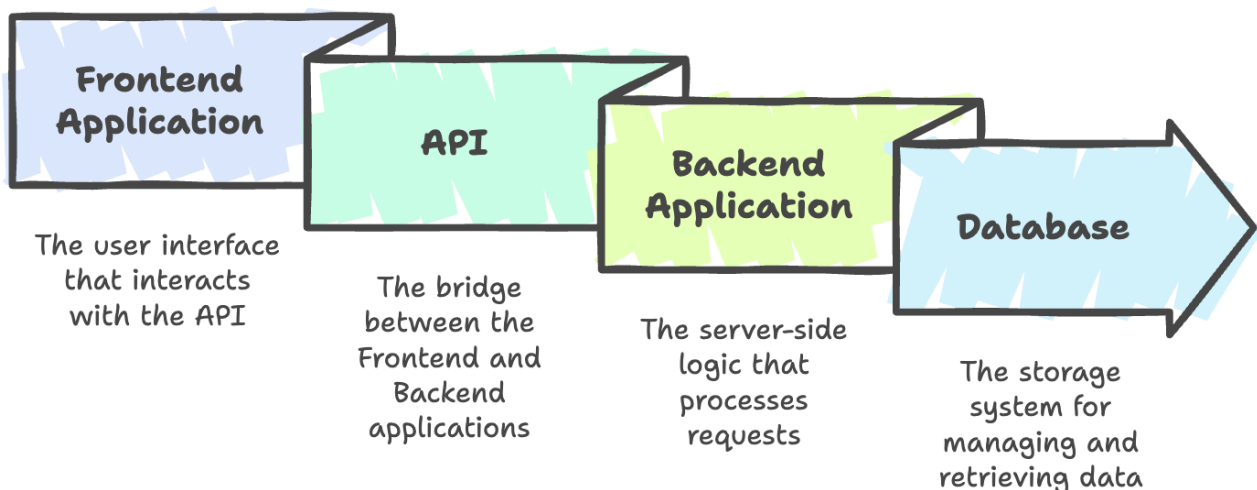
Caching in the whole process of DNS Resolution. There are zones for subdomain resolution in ANS part.

Cache Levels (checked in order)

Level	Location	Description
1	Browser Cache	Browser stores recently resolved domains locally
2	OS Cache	Operating system maintains its own DNS cache
3	DNS Resolver Cache	The ISP's resolver caches results for all users

API's - Application Programming Interfaces:

It is a set of endpoints exposed by an application that allows other applications (or frontends) to communicate with it, access its data, and perform operations without needing direct access to the underlying database.



If we give access to applications or frontends without APIs then it can be dangerous for our data in DB.

So APIs restrict or control this access.

Types of APIs:

1. REST APIs - Representational State Transfer - data format JSON - uses HTTP methods.
2. SOAP APIs - Simple Object Access Protocols - data format XML - used for cross platform communication.
3. Graph QL - Graph Query Language - frontend can request multiple table's data in single query - data format JSON - solves issue of multiple requests (under-fetching) and over-fetching.
4. Web sockets - Real time - two way communication between client and server.
5. gRPC - google remote procedure call - data format protocol buffers - small in size so faster - micro service to micro service communication.

API Type	Data Format	Best For	Latency	Complexity
REST	JSON	General-purpose web APIs	Low	Simple
SOAP	XML	Legacy/enterprise systems	Moderate	High
GraphQL	JSON	Flexible frontend queries	Low	Moderate
gRPC	Protobuf	Microservice internal communication	Very Low	Moderate
WebSocket	Any	Real-time bidirectional communication	Real-time	Moderate

RESTful APIs:

These are standard format for transferring data from clients to servers, designing these depends upon resources.

endpoint = http method + path.

endpoints is location from which we can access on API.

Method - Defines the operation (GET, POST, PUT, PATCH, DELETE)

Path - The URL through which data is accessed or submitted

Requests in REST APIs:

Methods:

Method	Path	Purpose
GET	/users	Retrieve all users
GET	/users/{id}	Retrieve a specific user by ID
POST	/users	Create a new user (data sent in body)
PUT	/users/{id}	Replace the entire user record
PATCH	/users/{id}	Partially update specific fields
DELETE	/users/{id}	Delete a user

A REST API request consists of - method + path url + headers + body (optional)

headers - contains meta data, tokens, permissions for the request.

path url - tells api which resource to access.

Some more concepts:

1. Nesting - When entities have relationships. eg - /blogs/{id}/comments
2. Filtering - uses query parameters to narrow down results based on conditions. eg - /comments?post_id=123

There are three ways to send data in a REST API request:

Method	Use Case	Visibility	Example
Path Parameter	Pass IDs, slugs, or unique identifiers	Exposed in URL	<code>/users/42</code>
Query Parameter	Filtering, sorting, pagination	Exposed in URL	<code>/users?age=20&sort=name</code>
Request Body	Sensitive or complex data (credentials, payloads)	Hidden from URL	<code>{ "username": "muskan", "password": "***" }</code>

Responses in REST APIs:

corresponds to what server sends for each client request.

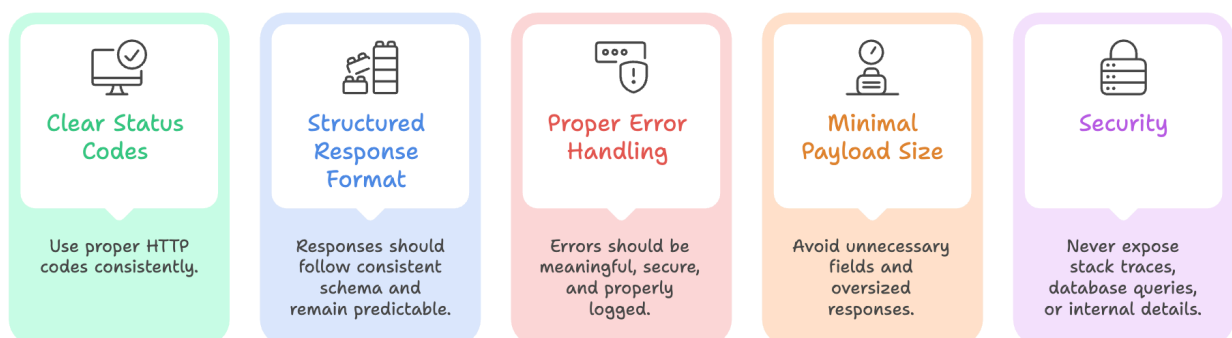
consists of status code + body returned.

status codes - for real better understanding and documentation clarity.

body returned - the actual response from server.

①	1xx	: Information (Ready to receive body, protocol changes)
②	2xx	: Success (ok, new resource created, no content)
③	3xx	: Redirection (moved permanently, temporary redirect, not modified)
④	4xx	: Client error (wrong data, unauthorized, not found)
⑤	5xx	: Server error (internal server error, not implemented, bad getu)

Characteristics of Good API Responses



Idempotent Requests:

This is very important in distributed systems as same requests can go through multiple times due to network partitions.

For an Idempotent request, when making same request for multiple time it must produce same business effect.

GET and PUT are generally idempotent.

POST is non idempotent naturally, but can be made by assigning unique keys to each operation.

Polling:

Client repeatedly asks server whether something changes or not.

useful when cheap solution is required and delay is acceptable, occasional updates.

Server Side Events:

This allows server to actively send update to client of something changes.

useful for notification sending, alarming something, live dashboard.

Web sockets:

Creates persistent bidirectional channel for communication. These are complex and expensive.

useful in games, chat etc.

Web hooks:

This is a technique to communicate between different services.

example when payment is done payment handler interface fires a web hook to inventory manager.

Databases:

do it from DBMS notes for detailed explanations.

Database Scaling.

Scaling reads - Partitioning

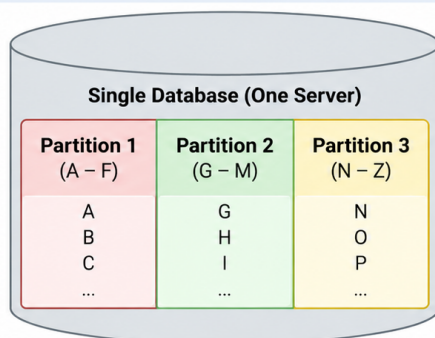
Scaling writes - Sharing

Sharding: method of distributing data across multiple machines

Partitioning: splitting a subset of data within the same instance

Partitioning

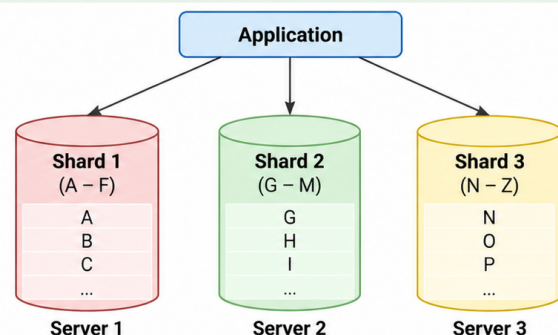
Partitioning splits a single large database (or table) into smaller parts (partitions), but all partitions reside on the **same server/instance**.



- Data is split into partitions (e.g., by range, hash, or list).
- All partitions are stored on the same database server.
- Helps with manageability and query performance.
- Does not scale to multiple machines (still one server).

Sharding

Sharding splits a large database into smaller, independent databases (shards) that are distributed across **multiple servers/machines**.



- Data is split into independent shards (e.g., by range, hash, or tenant).
- Each shard is stored on a different server/machine.
- Helps with horizontal scalability (can handle more data and load).
- Each shard works independently.

Caching:

What is cache?

Any memory/data that lets us avoid computation, network I/O, disk I/O is known as cache.
Without caching - load on server increases, more latency to users.
Generally stored as Key-Value pairs.

These can be at certain places:

frontend - client side cache - browser cookies.

backend - server side cache - remote cache.

database - takes some extra space - new column for aggregated data to be used as cache.

Some other terms:

cache hit - when the requested data is already present in the cache.

cache miss - when the requested data is not available in the cache.

TTL - Time to live (the time for which an entry will be auto deleted from cache)

Strategies of keeping updated cache:

1. Read Through Cache (RTC) - read from cache, write to DB and in case of cache miss out to cache.
2. Write Through Cache (WTC) - read from cache, write in cache and cache writes to DB. eg - stock prices
3. Write Around Cache (WAC) - read from cache, write always to DB. eg - Instagram posts.
4. Write Back Cache (WBC) - write to cache but write to DB is asynchronous so performance > consistency.

Strategy Comparison				
Strategy	 Read-Through	 Write-Through	 Write-Around	 Write-Back
Write Target	Database	Cache → Database (sync)	Database	Cache → Database (async)
Read Target	Cache	Cache	Cache	Cache
Consistency	Moderate	Strong	Low (risk of stale data)	Eventual
Best For	Read-heavy workloads	Data requiring latest values	Infrequently read after write	High write traffic, speed-critical

Cache Eviction Policies:

1. Least Recently Used
2. Most Recently Used
3. Least Frequently Used

4. First in First out
5. Last in First out

Cache Eviction Policies					
Policy	 LRU	 MRU	 LFU	 FIFO	 LIFO
Evicts	Least recently accessed	Most recently accessed	Least frequently accessed	Oldest inserted entry	Newest inserted entry
Based On	Last access time	Last access time	Total access count	Insertion order	Insertion order
Best For	General-purpose caching	Data unlikely to be reused	Workloads with clear popularity	Simple, predictable eviction	Specific use cases

Content Delivery Network (CDN):

These are a set of servers distributed across the world.

Request of client goes to the geographically nearest server.

Mostly configured as - lazy populating - means on cache miss data gets put from server once.

Generally used for static data, JS files.

Also used for providing region specific filtered data.

A **Content Delivery Network (CDN)** is a group of connected servers spread across the world that work together to load web content to users as fast as possible.  Cloudflare +1



How a CDN Works

- **Edge servers:** These are local servers placed in different regions.
- **Caching:** Edge servers save copies of website files like images, videos, and text.
- **Shorter distance:** When you open a website, a nearby edge server sends the files instead of the main origin server, which reduces wait time.
- **Less traffic:** Distributing the work keeps the main server from crashing when many people visit at once.  Amazon Web Services ... +5

Remote Cache (Redis):

Most commonly used central level cache. Multiple APIs access this cache over network call.

There should always be expiration time set for these caches (TTL) or memory leaks occur.

Database Caching:

After a heavy query is executed for the very first time, we can store it as new column for future use case.

Load Balancer:

It is a component that delegates the incoming client's requests to the most appropriate server base on algorithms to improve fault tolerance, performance, scalability and availability.

Types:

based on layer ->

1. layer 4 - transport layer, fast, content packet is not impt,
2. layer 7 - application layer, slow, content packet is impt,

based on routing ->

1. static - routes based on pre defined algorithms
2. dynamic - routes based on server's heath checks and performance, difficult to configure.

Tasks:

1. Delegation - routing to server in most efficient way.
2. Health Checks - should know which server has ability at present.

Algorithms for delegation:

Generally multiple approaches are combined together for best outcomes.

1. Round Robin - equal distribution, one by one request, issue when servers are not equally capable.
2. Weighted Round Robin - weight assigned on each server based on configuration of the server.
3. Geo-Based - req routed based on geographic location of server, can be mislead with VPNs, overhead of database replication to each location.
4. Least Connection Algo - number of connections, but not a good matrix when servers have different capacity.
5. Least Time - routes to lowest average response time, overhead of maintenance of reqs/sec. ideal for stocks system
6. IP Hash - client's IP is hashed to check for which server to route, same client same server -> can be cached, challenging to add new server -> rehashing needed.

Algorithm	Basis of Decision	State Maintained	Best For
 Round Robin	Sequential order	Counter only	Equal-capacity servers
 Weighted Round Robin	Server capacity	Weights	Heterogeneous server configurations
 Geo-Based	Geographic proximity	Location data	Globally distributed users
 Least Connection	Active connections	Connection count	Variable request durations
 Least Time	Response time	Avg response time	Latency-sensitive applications
 IP Hash	Client IP hash	Hash mappings	Session persistence

Health checks:

Health checks are mechanisms used by a Load Balancer to continuously monitor the availability, responsiveness, and overall health of backend servers.

By doing this we know about availability, reliability of response, user experience, metrics for future scalability.

Parameters:

1. Interval - Frequency at which health checks are performed.
2. Timeout - Maximum time the load balancer waits for a response.
3. Threshold - Number of consecutive successes or failures required before changing server status.

Types:

1. Passive Health Checks - No extra requests, checks from the real users responses.
2. Active Health Checks - dedicated requests are sent for checks, accurate but adds a small amount of traffic as well.

Database Replication:

Replication is the process of copying and maintaining data across multiple databases or nodes. Instead of storing data in a single location, identical copies are maintained on multiple servers.

Done for high availability, fault tolerance (avoids SPOF - single point of failure), high read throughput.

Types of Replication:

1. Single Leader Replication - one leader, others are followers, only leader accepts write requests, async replication (can be done sync as well), this may give faster responses but has temporary consistency issues for the time DB is not updated. In case of leader failure -> the replica having latest updated is elected as leader.
2. Multi Leader Replication - multiple data centres each having their own leader, leaders are given priority based on which data consistency is maintained.
3. Leaderless Replication - every node can receive read and write requests,

Handling failures:

Leader election policies:

heartbeats are monitored for health of leader.

for a follower database to become leader - Quorum / majority is important.

Term	Meaning
Leader	Node currently responsible for coordinating/ performing leader duties
Follower	Node that follows the leader and can potentially become leader
Candidate	Node participating in an election to become leader
Election	Process of selecting a new leader
Heartbeat	Periodic message used to show that the leader is alive
Election Timeout	Time after which lack of heartbeat can trigger an election

Majority / Quorum	More than half of nodes required for a valid election/decision
Term / Epoch	Logical period identifying a leadership generation
Failover	Switching to another node after failure
Split Brain	Multiple nodes/groups incorrectly believe they are the leader
Fencing	Preventing an old/stale leader from performing operations
Consensus	Distributed nodes agreeing on a decision/state
Raft	Consensus algorithm with an explicit leader-election mechanism
Paxos	Another family of consensus algorithms

Circuit Breaks:

A Circuit Breaker is a resilience pattern that stops sending requests to a failing/unhealthy downstream service for a period of time, preventing repeated failures and protecting the rest of the system.

Partitioning:

Partitioning is the process of dividing a large dataset into smaller, manageable chunks called partitions, which are distributed across multiple nodes.

Improves scalability (distributed storage) and performance (by parallelism in query execution).

Problem - when one partition receives more requests it becomes hotspot.

Partitioning Strategies:

1. Partition by Key - based on range of key values. hotspots when traffic is unevenly distributed -
2. Partition by Hash of Key - data stored in one partition has same hashed value.
3. Partition by local secondary index - each partition has it's own secondary index, so when data needs to be fetched we must query ALL partitions.
4. Partition by global secondary index - 1 centralised index for all partitions, here only required partitions are queried, NOT ALL.

Strategy Comparison

Strategy	Distribution	Hotspot Risk	Read Efficiency	Write Efficiency
 Key Range	Based on key ranges	High (skewed access)	Good for range queries	Simple
 Hash of Key	Based on hash values	Low (uniform)	Good for point queries	Simple
 Local Secondary Index	Per-partition indexes	Moderate	Scatter-gather (all partitions queried)	Simple
 Global Secondary Index	Centralized index	Low	High (targeted queries)	Complex (index update overhead)

CAP Theorem:

A distributed system cannot simultaneously guarantee all three properties: Consistency (C), Availability (A), and Partition Tolerance (P). It can guarantee only two of these three properties at any given time.

Property	Full Form	Meaning
C	Consistency	Every user always receives the latest and most up-to-date data from the system.
A	Availability	Every request receives a response, even if some nodes are experiencing failures.
P	Partition Tolerance	The system continues to operate even when communication failures or network partitions occur between nodes.

CAP Theorem Combinations

Combination	Guarantees	Sacrifices	Example
 CA	Consistency + Availability	Partition Tolerance	Small single-database apps
 CP	Consistency + Partition Tolerance	Availability	Banking, financial systems
 AP	Availability + Partition Tolerance	Consistency	Instagram, social media

When keeping partition we can have either consistent or available databases:

1. CA - SQL, these types of databases can't be distributed, as to keep C, A when P it must be single node.
2. CP - Mongo DB, turns off nodes when partition occurs -> availability destroyed.
3. AP - Dynamo, Apache Cassandra, in case of partition, let nodes be inconsistent, they become eventually consistent.

Message Queue (MQ):

A Message Queue is a communication mechanism that *enables asynchronous communication* between applications or services. It acts as an intermediary (buffer) that temporarily stores messages sent by one service until another service is ready to process them. Instead of requiring two services to communicate directly, the producer sends messages to the queue, and the *consumer retrieves/requests* and processes them independently.

Message queues improve scalability, reliability, fault tolerance, sudden traffics and *service decoupling* in distributed systems. eg - SQS, RabbitMQ

There are 2 types of communications:

1. *Synchronous* - we have to wait for response - finance apps, security, DB transactions.
2. *Asynchronous* - no waiting - email service, event driven calls, analytics pipelines.

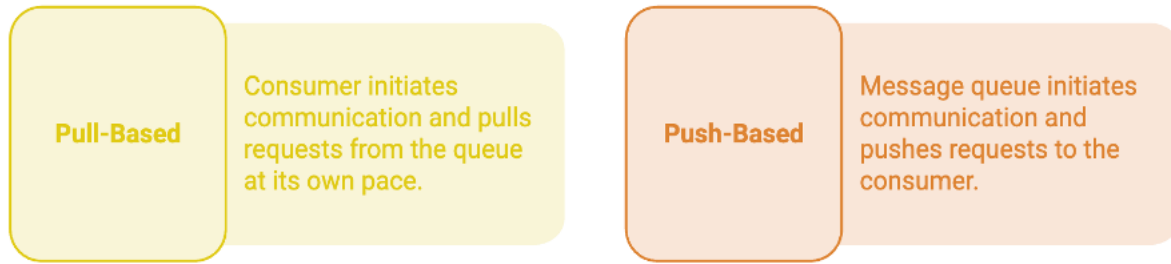
Participants:

1. Producer / Publisher
2. Consumer / Subscriber
3. MQ / Broker

Processing Approaches (queue -> FIFO):

1. *Strict Order Queue* - waits for failed req to resolve then moves to next one.
2. *Unordered Queue* - skips the failed req and moves to next one.
3. *Priority Queue* - high priority are processed before low priority.

Message Queue Models



Message Streams:

Streams are same as queues but with *heterogeneous consumers*. eg - Kafka, AWS Kinesis. Messages in streams lie forever (can be configured) and services iterate over the requests in these streams.

Kafka Essentials:

Topic: A logical category/stream in Kafka where related messages are published.

Partition: A topic is divided into ordered partitions, allowing messages to be distributed and processed in parallel.

Consumer Group: A group of consumers that collectively consume a topic, with each partition assigned to at most one consumer within the group.

Consumer == Partitions made.

Kafka partitions allow a topic to be distributed across brokers and consumed in parallel. This provides horizontal scalability and higher throughput while preserving ordering within each partition.

Pub-Sub Model:

This consists of multiple producers to publish and multiple consumers to consume independently.

Steps:

1. Publishers send messages to the queue.
2. The queue stores the messages.
3. Subscribers consume the messages independently.
4. The queue monitors subscriber health.
5. Failed deliveries are retried automatically.

Key factors for pub-sub:

Factor	Description
Message Order	Can be maintained using FIFO or Priority Queues depending on the implementation.
Message Consumption	Subscribers consume messages independently, and processing order may vary.
Poison Message	A message that repeatedly fails processing due to invalid data, bad requests, or consumer issues.
Duplicacy	Successfully processed messages must be removed from the queue to prevent duplicate processing.

Dead Letter Queue (DLQ):

Secondary queue to store messages that have been retried after multiple fail attempts. This prevents poison messages from blocking the main queue, easy to debug and troubleshoot, manual testing can be done.

Faults and Errors:

A fault or error occurs when a system fails to perform as expected or deviates from its intended behaviour. Faults can affect a system's reliability, availability, performance, and data integrity.

Categories of Faults:

1. Hardware - physical infrastructure of a system - random, unpredictable, hard to prevent - as hardware wears out at some time redundancy and data replication are major solutions. eg - disk full / crash, cable, server damage.
2. Software - issues in application code, system configuration, deployment, or software logic - edge cases, deployments, merge conflicts - preventable by unit testing, code reviews, exception handling, CI/CD.
3. Human Errors - incorrect actions or decisions made by users, developers, administrators, or operators - most unpredictable sources of system failures - prevented by RBAC, documentations, automation of repetitive tasks.

Category	Nature	Predictability	Mitigation
 Hardware	Physical infrastructure	Random	Redundancy, replication, monitoring
 Software	Application level	Identifiable	Code reviews, testing, proper handling
 Human	Human actions	Unpredictable	Automation, access controls, training

Monitoring and Observability:

These are practices used to continuously track the health, performance, and behaviour of a system.

Helps devs to quickly detect and resolve issues but is only useful when proper alerting is done.

Monitoring	Observability
Tracks predefined metrics, logs, and alerts	Helps understand the internal state of the system using metrics, logs, and traces.
Answers "Is something wrong?"	Answers "Why is something wrong?"
Focuses on known errors/problems	Focuses on unknown errors/problems
Common metrics: CPU utilisation, API response time, memory consumed, error rates, server capacity.	Common metrics: complete cycle of request to response through different micro services.

API Monitoring:

focuses on performance, availability and reliability of application endpoints (apis).

Components to be monitored ->

1. Throughput - the number of requests per unit time. determines load and scaling decisions.
2. Error Monitoring - tracks response codes for unsuccessful requests. determines failures in app & deployments
3. Health Checks - continuously verify if a service or component is healthy or not. determines if load balancer should route request to it or not, reliability.
4. Latency - the time required for a system to respond to a request. determines user experience and performance.
5. Bandwidth - data transferred per unit time.
6. Bottleneck - limiting capacity of any component becomes bottleneck for the system. eg - limited DB connections, external API usage.

More on Latency Estimations -

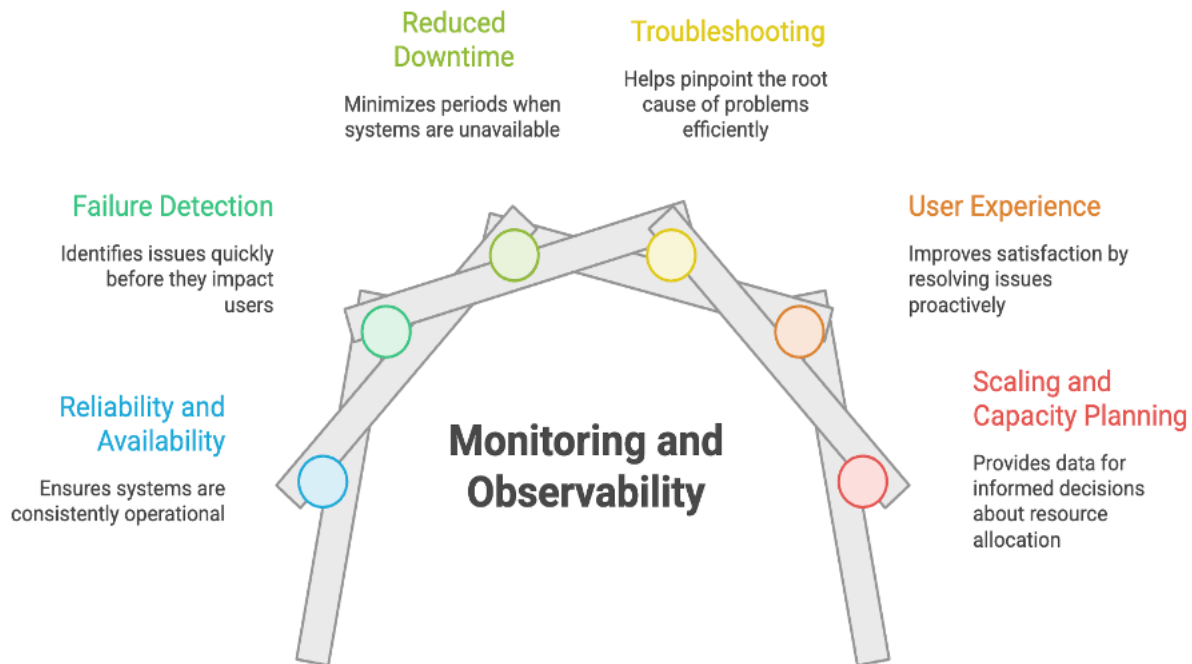
Averages are not true measure therefore we measure in percentiles.

A P90 latency of 200 ms means:

- 90% of requests complete within 200 ms (can be less as well).
- 10% take longer than 200 ms.

System Monitoring:

1. CPU Usage - determines performance bottlenecks, notify when set threshold is met.
2. Memory Usage - tracks RAM consumption, determines out of memory, resource allocation.
3. Disk I/O - read and write operations monitoring, determines storage and slow DB operations.
4. Network I/O - incoming and outgoing traffic, determines cyber attacks, communication b/w services.



Bloom Filters (Bit array):

These are approximate/probabilistic data structures that say with 100% certainty. Naive way: keeps track of everything a user sees - so to implement something like this: we have to load set in memory and that expensive and has high time complexity.

we do not actually store data we just store the modulo of hashed string: so very space efficient. As we have to choose a fix size of this bit array -> false positive rate increase with time but always 0 false negative rate, also we cannot increase size later as the previously added things will completely go wrong -> same drawback of hashing everywhere.

Bloom Filter

A space-efficient probabilistic data structure to test whether an element is (possibly) in a set.
Supports fast membership queries with no false negatives but possible false positives.

1. What is Bloom Filter?

- A bit array of size m (all bits initially 0).
- Uses k independent hash functions.
- To insert an element, set k corresponding bits to 1.
- To check membership, if all k bits are 1 $\rightarrow$ element is possibly present.
- If any bit is 0 $\rightarrow$ element is **definitely not present**.

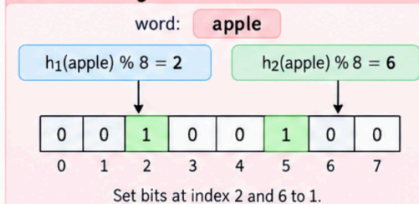
2. Structure

Bit array (size m)

0	0	1	0	0	1	0	0
0	1	2	3	4	5	6	7

m = size of bit array (e.g. 8 bits)
 k = number of hash functions (e.g. 2)

3. Inserting an Element



4. Checking Membership

Query: **apple**

- Compute $h_1(\text{apple}) \% 8 = 2$ and $h_2(\text{apple}) \% 8 = 6$.
- Check if bits at 2 and 6 are 1 $\rightarrow$ **Both 1 $\rightarrow$ possibly present.**

Query: **dog**

- Compute $h_1(\text{dog}) \% 8 = 5$ and $h_2(\text{dog}) \% 8 = 0$.
- Check if bits at 5 and 0 are 1 $\rightarrow$ bit at 5 is 0 $\rightarrow$ **definitely not present.**

5. Example

Insert: **apple** **ball** **cat**

$h(\text{apple}) \% 8 \rightarrow 2$
 $h(\text{ball}) \% 8 \rightarrow 6$
 $h(\text{cat}) \% 8 \rightarrow 2$

0	0	1	0	0	1	0	0
0	1	2	3	4	5	6	7

Query **dog** $\rightarrow h(\text{dog}) \% 8 = 5$
bit[5] = 0 $\rightarrow$ **dog not present.**

6. False Positive

Insert: **dog** **elephant**

$h(\text{dog}) \% 8 \rightarrow 5$
 $h(\text{elephant}) \% 8 \rightarrow 6$

0	0	0	0	0	1	1	0
0	1	2	3	4	5	6	7

Query: another word (e.g. lion)
If $h(\text{lion}) \% 8 = 5$ and 6, both bits are 1 $\rightarrow$ reported as present even if lion was never inserted.
This is a false positive.

7. False Negatives

- Bloom filter never gives false negatives.
- If an element was inserted, all its k bits will always be 1.

8. Why False Positive Rate Increases?

- As more keys are inserted, more bits are set to 1.
- Bit array becomes crowded.
- New elements are more likely to map to already set bits. $\rightarrow$ Hence false positive rate increases.

9. Handling Increasing Number of Keys

- Re-create a larger Bloom filter (larger bit array size m).
- Re-insert all existing keys (recompute hash values).
- Estimate expected number of keys and provision a large enough bit array initially.

Key Takeaways

- Space efficient
- Fast queries ($O(k)$)
- No false negatives
- Possible false positives
- Larger bit array & optimal k reduces false positive rate

Consistent Hashing:

Used for knowing "who owns the data".

Consistent Hashing

A distributed hashing technique that **minimizes data movement** when the number of nodes (servers) changes. It solves the **data ownership** problem and is widely used in load balancers, caching systems, and distributed storage.

1. Why Consistent Hashing?

- In normal hashing, when we add/remove a server, many keys need to be rehashed and moved.
- Consistent hashing minimizes the number of keys that need to move.
- It solves the **Data Ownership problem**:
→ each key is mapped to a specific server.
- It is one of the most popular algorithms for distributed systems.

2. Key Idea

- Arrange all nodes (servers) in a circular hash space (ring) of size 2^m (e.g. 0 to $2^{128} - 1$).
- Use a hash function (e.g. SHA-256 / SHA-128).
- Hash each node's identifier and place it on the ring.
- To find the server for a key, hash the key and move clockwise to the next node on the ring.
- Each node is responsible for the range of keys between the previous node (exclusive) and itself (inclusive).

3. Hash Ring Visualization

The ring represents the hash space (e.g. 0 to $2^{128} - 1$).

Each node is responsible for the keys in the range (from previous node, to itself).

4. Mapping Keys to Nodes

key = "apple"
hash(key) = 2

Move clockwise to the next node (Node A). So, "apple" is stored on Node A.

5. Example with Multiple Keys

Key	hash(key) % 16	Mapped Node
apple	2	Node A (3)
ball	6	Node B (8)
cat	10	Node C (12)
dog	5	Node B (8)
elephant	6	Node B (8)

Note: If a key hashes exactly to a node, it maps to that node. Otherwise, it maps to the next node in clockwise direction.

6. Adding a New Node (Scaling Up)

Only the keys in the range (D, E) need to move to the new node. All other keys remain on their existing nodes.

Minimal data movement ✓

7. Removing a Node (Scaling Down)

All keys in the range (D, A] will now be moved to the next node (Node B). Other keys remain unaffected.

Only a small fraction of keys move ✓

8. Hash Function

- Commonly used: SHA-256 or SHA-128.
- Produces a large integer (e.g. 0 to $2^{128} - 1$).
- Given hash functions are cyclic, so we can visualize the hash space as a ring.
- Every node occupies one slot in the ring, where the slot is the hash of its identifier (e.g. IP + port).
- Each key is assigned to a node by passing its identifier through the hash function and moving clockwise.

9. Important Points

- Load balancers are stateless (use hash of access token to decide backend server).
- If one server goes down, routing changes and requests are distributed among remaining servers (no hiccups).
- When nodes change (add/remove), only a small subset of keys are re-evaluated and moved.
- Helps in minimal data transfer.
- The ring can be implemented using a simple array (e.g. sorted list of node hash values + binary search).

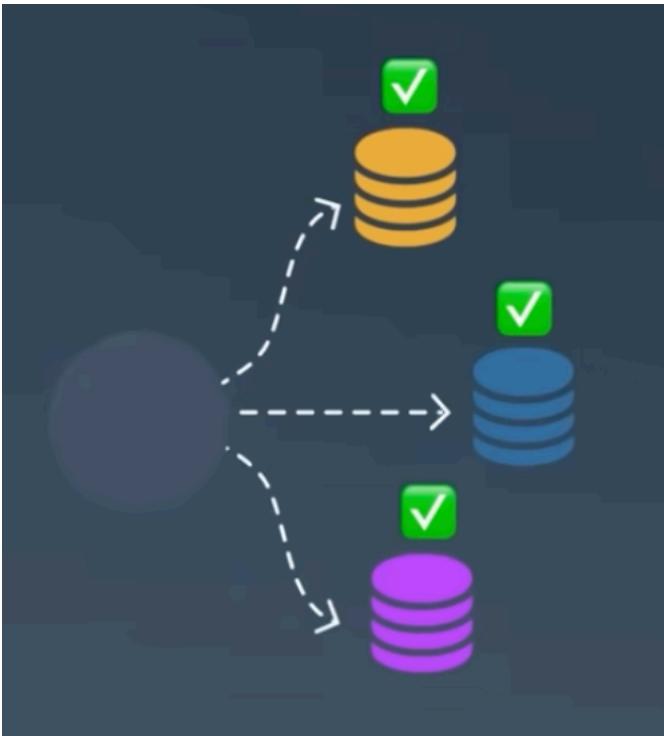
Key Takeaways:

- Solves data ownership problem
- Minimal data movement
- Used in load balancers, caching, distributed storage
- Nodes arranged in a hash ring
- Add/remove nodes with minimal impact

Distributed transactions:

When a payment happens and we book seat, payment update and record log -> we made this into 1 transaction.

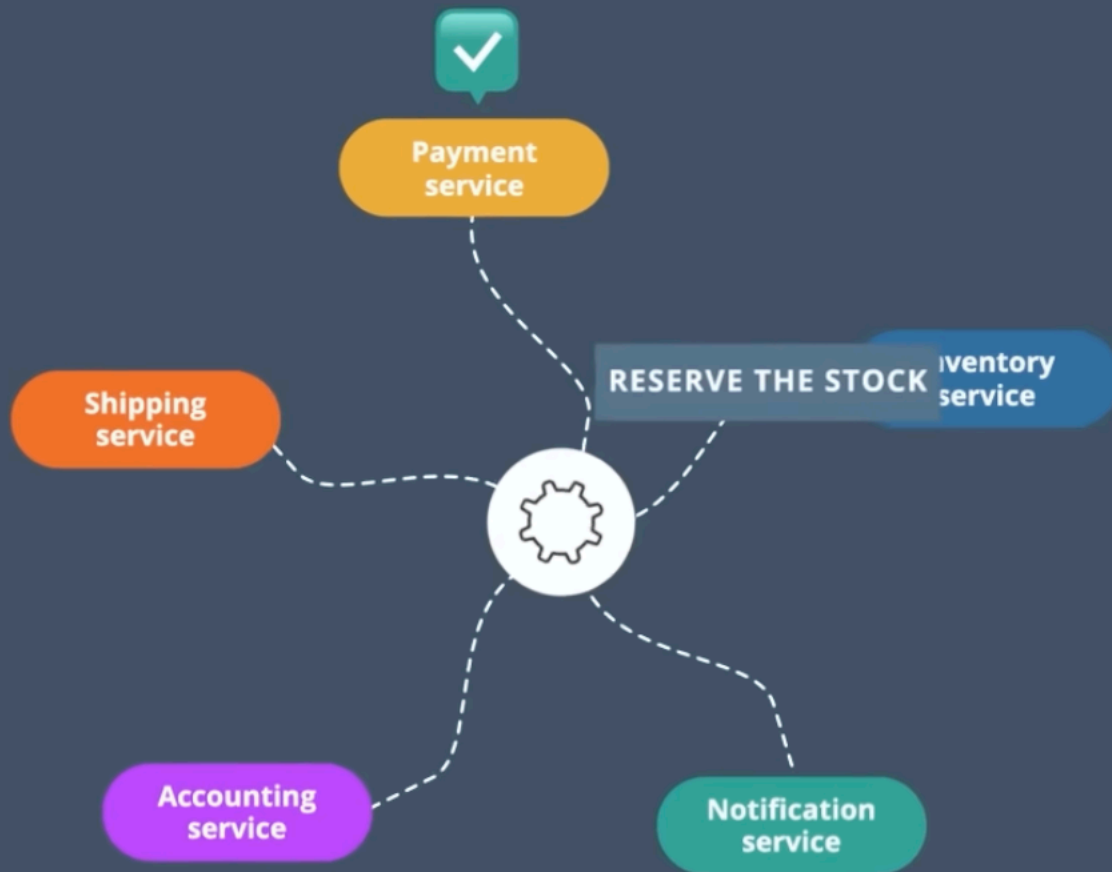
But what if we have scaled to a level where we have to make these different services -> we have to use distributed transactions.



Two ways to implement distributed transactions are:

1. 2PC - two phase commit, blocking protocol - not used at scale,
2. Saga Pattern - if one of services fail we do not roll back instead we just make compensating action. Eventual consistency is reached. two ways - Choreography and orchestration

Orchestration



Big Data:

When one machine is not capable enough to process data we do divide and conquer- it's called processing big data.

Companies do this for extracting insights, train models, move across data bases - they use big data tools.

Suppose we have to calculate word frequency:

approach 1 -> $O(n)$ traversal and map.

approach 2 -> we have multiple cores we can simply use multiple threads and parallelise the task.

but what if the data can't fit in 1 machine?

we make 1 machine coordinator, apply approach 2 in individual machines and all machines report the result to the coordinators.

what if any machine crashes, scalability concerns, reliability of results?

this is what big data tools take care off.

we just write the business logic.

Big Data Tools manages these complexities for us

- distribute across machines
- knowing which machines are doing what
- retrying in case of failures
- reprocessing in case of crashes or corruption
- cleaning up the resources once job is complete

Other topics to be covered:

Rate Limiting - Token Bucket, Leaky Bucket, Fixed Window, Sliding Window, Distributed rate limiting

Consistent Hashing ✓

Consistency Models - strong ✓ eventual ✓ RAW monotonic casual

Proxies ✓ & API Gateways

CDN ✓

monolith architecture

Idempotent meaning and requests ✓

Optimistic vs. pessimistic locking ✓, distributed locks

polling ✓ and Server Sent Events ✓

Kafka

Edge Servers

Connection Pooling

Reliability Patterns - jitter timeout circuit break bulkhead

Database - b/b+ trees

Cache - stampede, avalanche penetration hotkey invalidation

Stateful and opposite

Disaster recovery

2PC in transactions ✓

CQRS

Event Sourcing

CDC - change data capture

Service mesh

Bloom filter ✓

vector clocks

kubernetes intervals

Saga ✓

